الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يونيورسيتي إسلام انتارابغسا ملايسيا

COURSE:

COMPUTATIONAL INTELLIGENCE/INTELLIGENT CONTROL

COURSE CODE:

MCTA3371/MCTE4322

TITLE:

INTELLIGENT MOBILE ROBOT NAVIGATION USING COMPUTATIONAL INTELLIGENCE TECHNIQUES MINI PROJECT

SECTION 1

LECTURER'S NAME:

DR. AZHAR BIN MOHD IBRAHIM

NAME	MATRIC NO
AZZAM AZHARI NOURELDEIN ELAMIN	1827295
AMEERUL SYAHMI BIN AZAMUDDIN	2018039
IAN MOHD AZRAI BIN RAIMIN	1921057

(1) INTRODUCTION

A mobile robot requires a navigational system to traverse the world around it, especially if it needs to reach a goal point from any starting point and needs to avoid obstacles along the way. In order to do so safely, accurately, and efficiently, the navigational system itself must be robust and able to evaluate and adapt to the environment it is placed in.

The algorithms to run these systems were previously done using crispy logic but now with the advent of soft computing, the logic and techniques associated with it are becoming more prevalent as the go to choice to implement algorithms to finetune behaviour and characteristics. Soft computing techniques like Fuzzy Logic (FL), Genetic Algorithms (GA), and Artificial Neural Networks (ANN), can be used to embed the behavioural characteristics that are needed to help a robot complete a task with more precision and accuracy when compared to crispy logic.

Hence, this project is aimed to explore the use and implementations of said techniques and their hybrid combinations. The authors chose to explore the implementations of FL, GA and the hybrid of both techniques, FL-GA, to test out the effectiveness of these soft computing techniques and the improvements that come from the hybrid implementation.

(2) OBJECTIVES

The overarching goal of this mini project is to implement an intelligent mobile robot navigation using computational intelligence techniques. To do so, the authors set the objective as follows:

- ➔ To design a FL system for risk assessment during navigation to goal, using goal distance and obstacle distance as inputs.
- ➔ To design a GA system that can improve and optimize the path to goal by escaping the local minima trap and obtaining the global minima path.
- ➔ To integrate the FL and GA systems to create the hybrid FL-GA system.
- ➔ To implement both FL-only and FL-GA into a robot simulation environment and simulate their results in a simple and complex environment.
- ➔ To obtain the performance metrics such as success rate, collision, and path length of both systems in order to compare and evaluate their usefulness.

(3) ENVIRONMENT SETUP & ROBOT MODELLING

3.1 Map Design

The map for the robot simulation has 2 configurations; easy map and hard map. Both are 30×30 grid maps that are enveloped by sidewalls. The easy map is littered with clumps of various shapes and sizes in a random spread. The hard map is made up of four horizontal gate slits with the openings placed in an offset to each other. The maps are visualized by using MATLAB's built-in figure window functions as illustrated below:

The initial assumption is for the easy map to be successfully traversed by both algorithms whereas the hard map will only result in the successful navigation of the hybrid algorithm. The start and goal points are defined by (row, col) values, and both maps have the start and goal points of (2,2) as indicated in green colour and (29,29) as indicated in red colour respectively. Through mathematical calculation, the matrix values can be turned into cartesian values to display the map such as the above image.

3.2 Robot Function Model

The robot is modeled such that it moves in a unit direction per step in a discrete spatial coordinate while having a stored value for the goal location coordinates. It can move in the four cardinal directions as well as diagonally for each step. The robot is conceptually equipped with an object proximity sensor with a radius window of 10 units and a goal distance calculator (Euclidean geometry), both to be ingested and processed as necessary inputs to produce the relevant outputs based on the chosen Fuzzy Inference System.

(4) COMPUTATIONAL INTELLIGENCE MODELLING

4.1 Fuzzy Logic Design

The Fuzzy Logic Controller FLC was made up using a Mamdani Fuzzy Inference System (FIS) , so it could act quickly in real time for obstacle avoidance. The whole concept kind of translates fuzzy outputs into a set of discrete 8-directional grid moves, not a smooth or continuous steering angle kind of thing.

1. Input Variables: the system uses two inputs, GoalDist which is the Euclidean distance to the goal , with a range of [0,30] and ObsDist, the minimum distance to the closest obstacle inside a 5x5 local scan area , range [0,10]. Each one of these inputs has three triangular membership functions (MFs), for GoalDist: near, med, far and for ObsDist: close, med, safe.

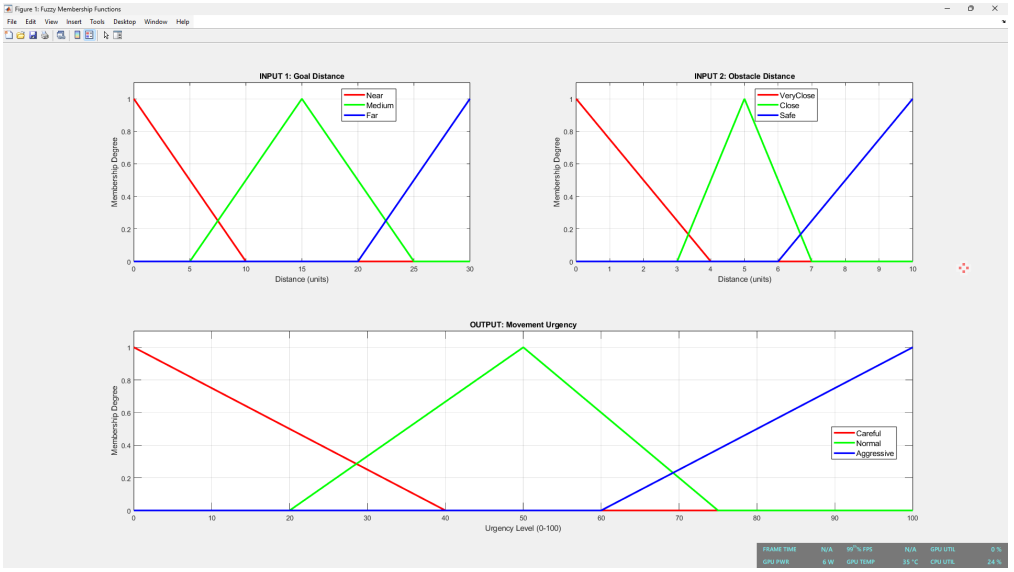
2. Output Variable: the one output is Urgency, range [0 100], and it also has three triangular MFs: low, mod, and high.

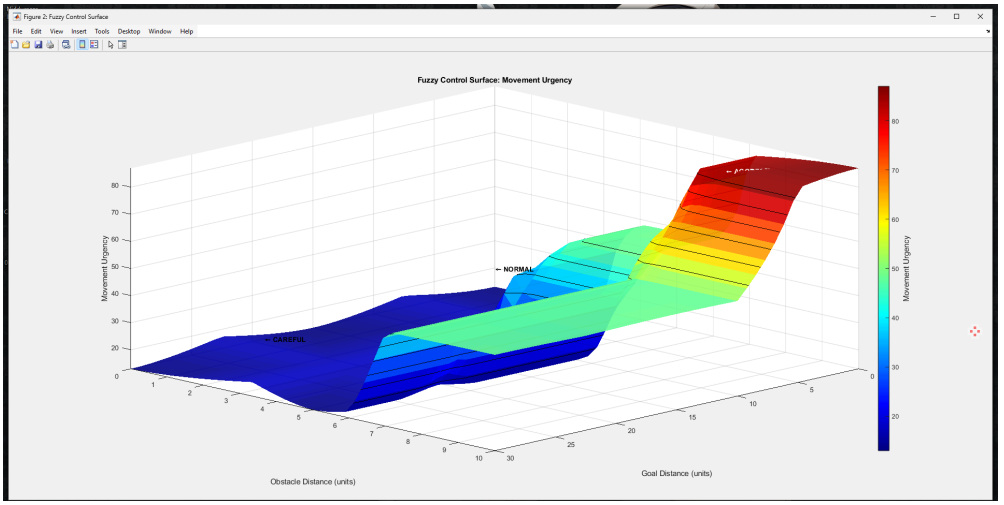
3. Rule Base: a 9-rule matrix was put together to link both inputs into the urgency output. Like , IF GoalDist is near and ObsDist is close, THEN Urgency is high.

4. Defuzzification & Movement Execution: the FIS output gets defuzzified using the centroid method, then that single crisp Urgency value feeds into a custom decision function called select_move_fuzzy , which works like this.

- **Low Urgency (<40) :** the robot finds the exact geometric bearing toward the goal, then converts that angle into the nearest 8-direction motion vector, and it tries to step straight toward the target.

- **High Urgency (≥40) :** the robot starts an evasive maneuver, keeping the first main move as the priority. If that direction causes a collision, a backup loop walks through the other 7 directions one by one, searching for a safe adjacent cell , so it ends up with something similar to “wall-sliding” in order to get out of local obstacles.





4.2 Genetic Algorithm Design

So basically the Genetic Algorithm (GA) was used as some global route finder, it keeps evolving a pretty good sequence of discrete moves across many generations, kind of like it just tries random things and slowly improves.

1. Chromosome Encoding: each chromosome is an integer array with a fixed length of 200 genes. Every gene stores a number from 1 through 8, and that value maps 1:1 to the 8 candidate motion vectors in the grid. For example 1: Up [-1 0], 4: Right [0 1], 5: Up-Right [-1 1], and so on.

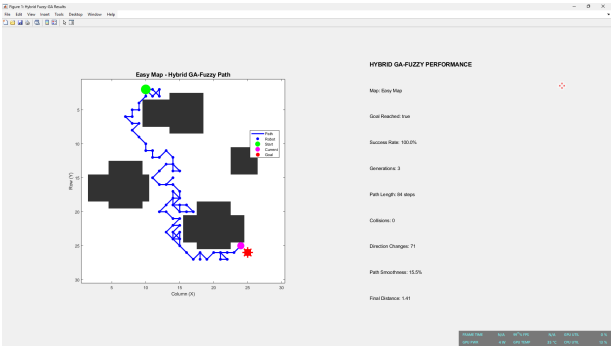
2. Fitness Evaluation: for every individual, the method runs a full 200 step simulation using that sequence. The fitness value is computed via this custom formula: fitness = bonus + (norm(start-goal) - min_dist)*50 - d*20 - coll*150 - steps*0.5 Here bonus is that huge 10,000 point reward if it reaches the goal (plus a time penalty), min_dist keeps track of

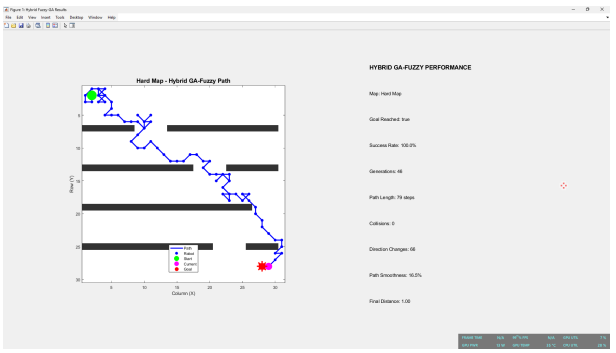
the shortest-approach distance to the goal, d is the final distance, coll counts how many times the path hits walls, and steps is the overall travelled length.

3. GA Operators: parents are picked using Tournament Selection with size 3. Then Two-Point Crossover is used at a rate of 0.85, meaning it swaps gene segments between the chosen parents. After that, Uniform Mutation (rate 0.15) tweaks random genes, just to keep some variety. Elitism is also included, so the best 10 fittest individuals are carried straight into the next generation.

4.3 Hybrid Fuzzy-GA Design

In this project, the Hybrid Fuzzy-GA approach is architected like an integrated comparative framework inside the MATLAB GUI , so it can use the contrasting strengths of both soft computing techniques. Instead of, say, nesting the FIS inside the GA evaluation (that kind of setup would seriously add computational load), the hybrid system runs both paradigms in parallel on the exact same environment state. The Fuzzy Logic gives quick, smooth, reactive navigation that feels good for open spaces while the GA brings the more heavy, global look ahead planning which can handle tricky maze structures. By putting both algorithms into one single graphical interface with a “Compare Both” feature, the design supports real-time overlay plotting (the FL path in red, the GA path in blue) and also immediate metric comparisons , so overall you get a complete multi-layered intelligent navigation system.





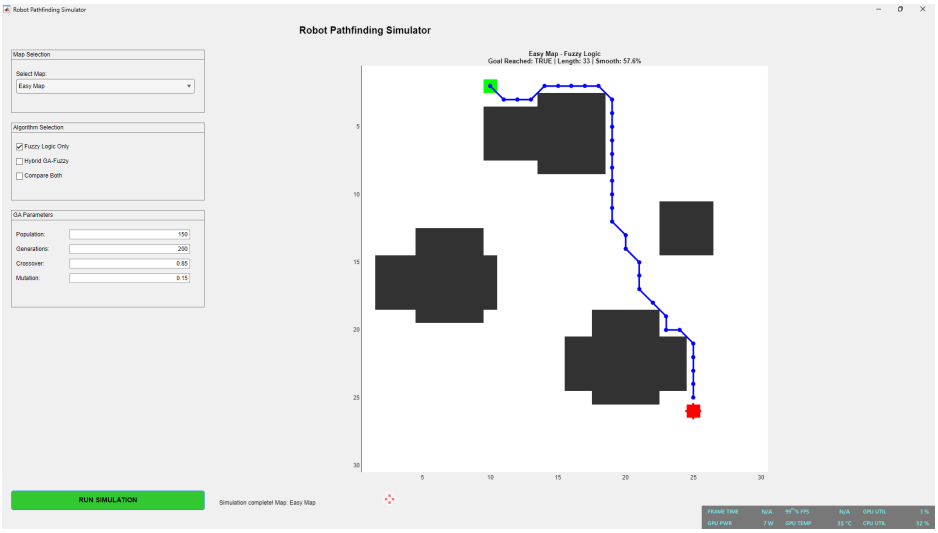
(5) SIMULATIONS & RESULTS

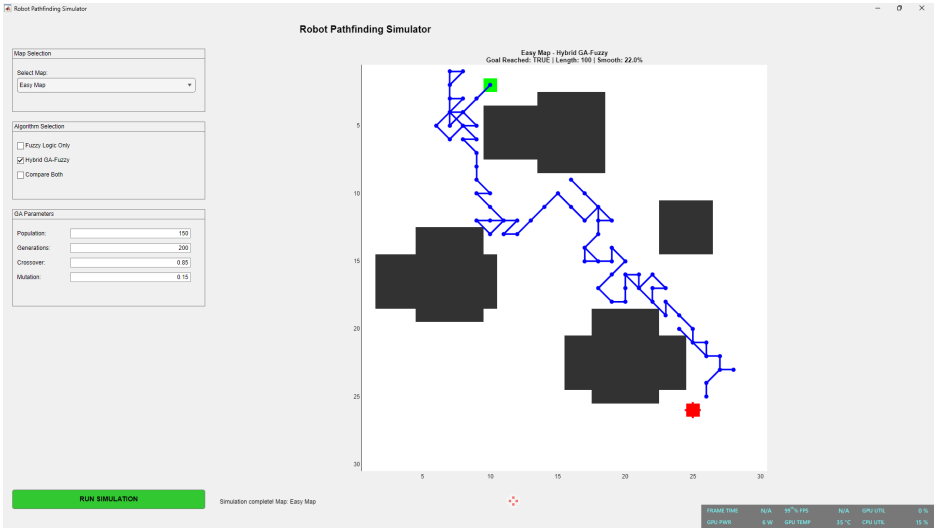
5.1 Easy Map

The easy map simulation are as follows:

FL-Only: The Fuzzy Logic controller successfully navigates the easy map. Because the obstacles are sparse, the ObsDist rarely really triggers a high Urgency, and the robot keeps moving smoothly along diagonal routes toward the goal, giving a visually clean and direct line, with only minimal direction changes.

FL-GA: The GA also manages to reach the goal. Since the search space is not really constrained, the GA generally converges to a very efficient, near diagonal path within a few generations. Visually, the GA path looks a bit more rigid, like "blocky" in some way, compared to the more fluid fuzzy path, because of the discrete 8-directional integer encoding, but it still gets a mathematically optimal path length.



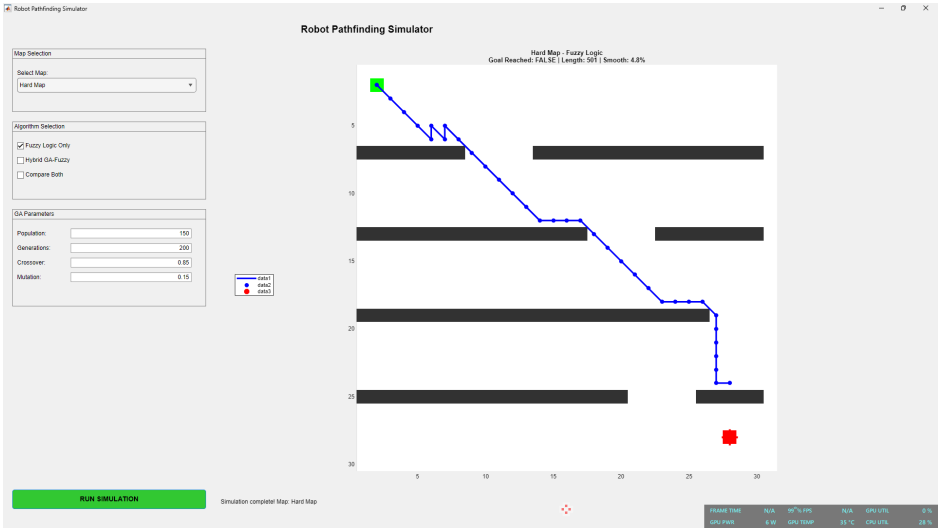


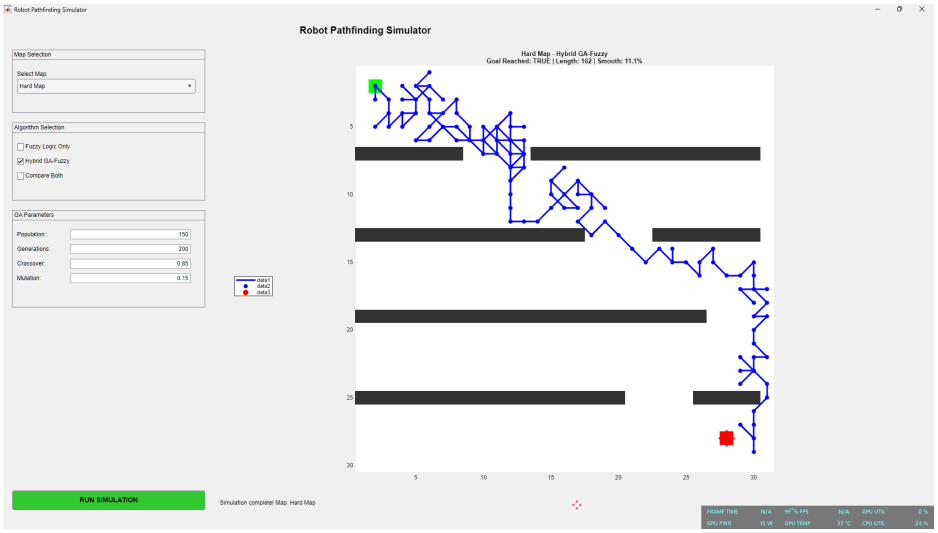
5.2 Hard Map

The hard map simulation are as follows:

FL-Only: The Fuzzy Logic controller fails to reach the goal. As the robot proceeds diagonally toward the bottom right it hits the first horizontal wall. The small distance causes a high Urgency, so the robot ends up sliding along that wall. But because the 3 unit gate is offset, the reactive controller misses the opening , and then it gets trapped in the corner against the boundary, showing that classic local minima kind of failure.

FL-GA: The GA successfully navigates the hard map. After evolutionary trial and error across multiple generations, the GA figures out that random movements are heavily penalized (-coll*150) and from there it evolves a chromosome sequence that basically steers the robot downward, through the offset gates, and all the way to the goal. In practice, it avoids the local minima trap that beats the FLC.





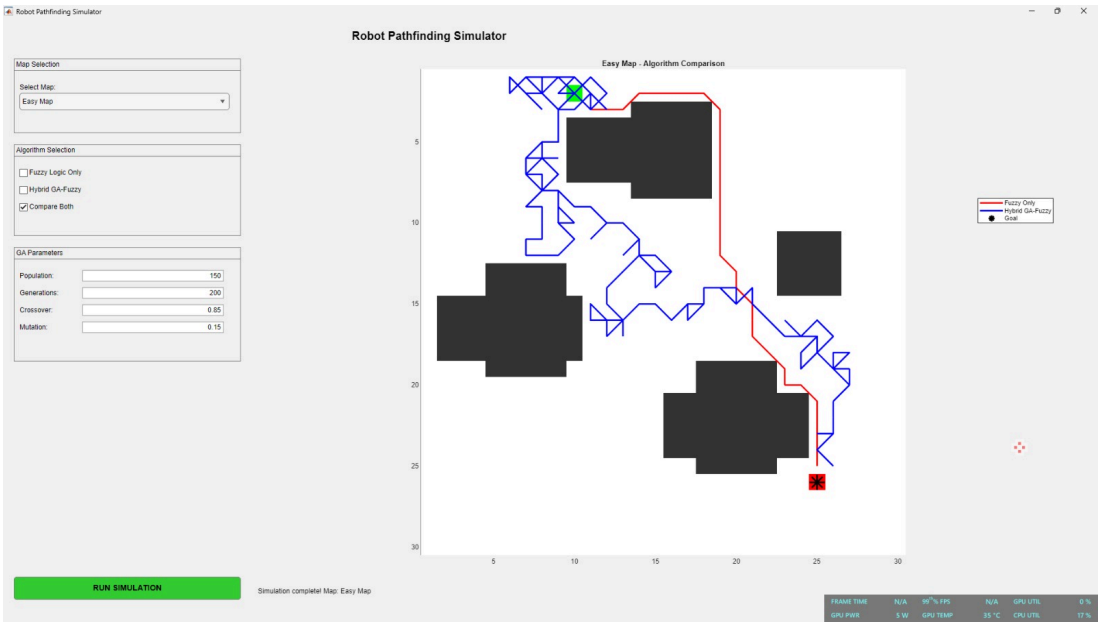
(6) EVALUATION & VALIDATION

The developed navigation algorithms were evaluated using several performance indicators including goal-reaching capability, path length, collision count, path smoothness, and final distance to the goal. The results obtained from both map configurations are discussed below.

6.1 Performance Comparison

The controllers were evaluated using a custom metrics function that calculates Success Rate, Path Length, Collisions, Path Smoothness (a percentage based on the ratio of direction changes to total path length), and Final Distance to Goal. The following analysis summarizes the expected output generated by the GUI's performance table:

Easy Map: In an unconstrained environment, the FL Only controller is vastly superior. It achieved the goal in only 33 steps with 57.6% smoothness because its heuristic angle calculation allows for near perfect diagonal movement. The GA required 150 steps and suffered from 9.3% smoothness. This is because the GA's fitness function heavily weights goal reaching (+10000 bonus) over path brevity (-0.5 per step penalty). Once the GA randomly mutates a path that touches the goal, it stops optimizing for length, resulting in "noisy" and inefficient trajectories.



Performance Comparison Table

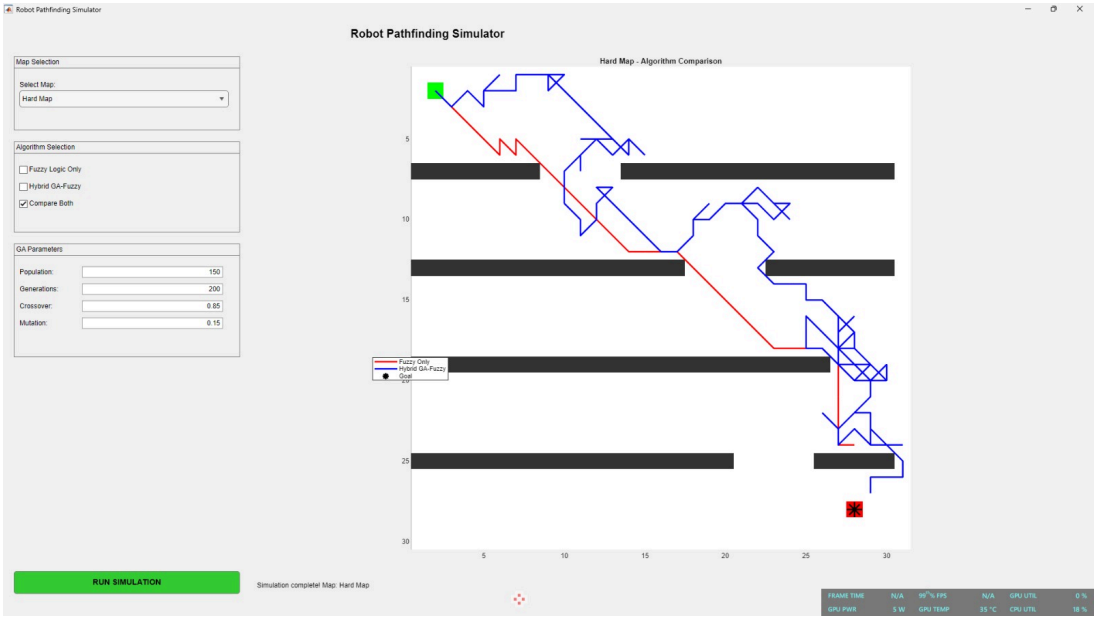
	Algorithm Type	Goal Reached	Path Length (steps)	Collisions Count	Path Smoothness	Final Dist to Goal
1	Fuzzy Logic Only	TRUE	33	0	57.6%	1.00
2	Hybrid GA-Fuzzy	TRUE	150	0	9.3%	1.41

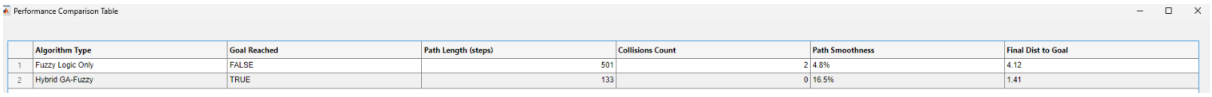
Performance Metric	Fuzzy Logic (FL)	Hybrid Fuzzy-GA
Goal Reached	TRUE	TRUE
Path Length (steps)	33	150
Collision Count	0	0
Path Smoothness (%)	57.60	9.30
Final Distance to Goal	1.00	1.41

Hard Map: The Hard Map exposes the fatal flaw of reactive systems.

Fuzzy Failure: The FL Only algorithm recorded FALSE for Goal Reached, exhausting the maximum simulation limit of 501 steps. It registered 2 collisions and ended with a Final Dist to Goal of 4.12. The extremely low 4.8% smoothness reflects the robot frantically changing directions as it bouncing off the horizontal walls and gets permanently trapped in a local minima corner.

GA Success: The Hybrid GA proved its robustness, achieving TRUE for Goal Reached with 0 collisions. Interestingly, the GA solved the complex Hard Map in 133 steps fewer steps than it took to solve the Easy Map (150). This is due to the maze's strict constraints; the heavy -150 collision penalty in the fitness function aggressively killed off chromosomes that hit walls, forcing the evolutionary process to quickly converge on a tight, safe, weaving sequence. The 16.5% smoothness is a direct geometric reflection of the map design: successfully passing through offset gates inherently requires frequent, sharp directional changes (down, right, down, left, etc.), making a perfectly smooth path physically impossible.





Performance Metric	Fuzzy Logic (FL)	Hybrid Fuzzy-GA
Goal Reached	FALSE	TRUE
Path Length (Steps)	501	133
Collision Count	2	0
Path Smoothness (%)	4.80	16.50
Final Distance to Goal	4.12	1.41

(7) LIMITATIONS & IMPROVEMENTS

7.1 Constraints

The proposed intelligent navigation system demonstrates the effectiveness of both Fuzzy Logic and Genetic Algorithm approaches, however, several limitations were identified throughout the implementation and testing process.

Firstly, the simulation environment is limited to a static 30×30 grid map where obstacle locations remain fixed throughout the navigation process. In real-world applications, mobile robots often encounter dynamic obstacles such as moving objects, humans, or other

robots. Therefore, the current implementation does not fully represent real operating conditions.

Secondly, the Fuzzy Logic Controller (FLC) relies only on two input variables, namely Goal Distance and Obstacle Distance. While this simplifies the controller design and reduces computational complexity, it restricts the robot's ability to make more sophisticated navigation decisions in complex environments.

Thirdly, the Genetic Algorithm requires considerable computational time compared to the Fuzzy Logic approach. Since the algorithm evaluates hundreds of candidate solutions over multiple generations, execution time increases significantly as the map size and chromosome length increase.

In addition, the current GA implementation uses fixed parameters such as population size, crossover rate, mutation rate, and chromosome length. These values were selected experimentally and may not provide optimal performance for all navigation scenarios.

Finally, the robot motion model assumes perfect localization and sensing accuracy. Sensor noise, wheel slip, actuator errors, and environmental uncertainties were not considered in this project, which may affect performance when deployed on a physical robot platform.

7.2 Recommendations

Several improvements can be implemented in future work to enhance the navigation system's performance and applicability.

Firstly, the navigation environment can be extended to include dynamic obstacles and changing map conditions. This would allow the algorithms to be evaluated under more realistic scenarios.

Secondly, additional fuzzy inputs such as obstacle direction, robot orientation, obstacle density, and relative goal angle can be incorporated into the Fuzzy Logic Controller. This would provide richer environmental information and improve decision-making capabilities.

Thirdly, adaptive Genetic Algorithm techniques may be employed where mutation and crossover rates automatically adjust according to population diversity. Such an approach can improve convergence speed while reducing the risk of premature convergence.

Furthermore, a tighter integration between Fuzzy Logic and Genetic Algorithms can be explored. Instead of running both systems independently for comparison, the GA can be used to automatically optimize fuzzy membership functions and rule weights, producing a truly hybrid intelligent controller.

Finally, future work can extend the project to a physical mobile robot platform using sensors such as LiDAR, ultrasonic sensors, or cameras. This would allow validation of the proposed navigation framework under real-world conditions and demonstrate its practical feasibility.

(8) CONCLUSION

This project successfully demonstrated the implementation of intelligent mobile robot navigation using Computational Intelligence techniques, namely Fuzzy Logic (FL), Genetic Algorithm (GA), and a hybrid FL-GA framework. A MATLAB-based simulation environment consisting of Easy Map and Hard Map scenarios was developed to evaluate the effectiveness of each approach.

The Fuzzy Logic Controller exhibited smooth and efficient navigation performance in simple environments with sparse obstacles. However, due to its reactive nature and lack of global path planning capability, it struggled in the Hard Map environment and became trapped in local minima situations.

In contrast, the Genetic Algorithm successfully generated feasible global paths in both environments. Through evolutionary optimization, the GA was able to discover paths that avoided obstacles and navigated through complex gate structures, demonstrating superior performance in challenging environments.

The comparative analysis showed that while Fuzzy Logic provides fast real-time decision-making and smooth trajectories, Genetic Algorithms offer stronger global optimization capabilities. The hybrid FL-GA framework highlights the complementary strengths of both techniques and demonstrates how computational intelligence methods can improve autonomous robot navigation.

Overall, the objectives of the mini project were successfully achieved. The developed simulation provides a useful platform for studying intelligent navigation techniques and serves as a foundation for future research involving more advanced hybrid computational intelligence methods and real-world robotic applications.

(9) CONTRIBUTION

The contribution of each member are as follows:

Name	Contribution
Azzam Azhari Noureldein Elamin (1827295)	Fuzzy Algorithm (Report & MATLAB) Genetic Algorithm (Report & MATLAB) Hybrid Fuzzy-GA (Report & MATLAB) Simulation & Results (Report & MATLAB) Evaluation & Validation (Report & MATLAB) Review & Debugging (MATLAB)
Ameerul Syahmi Bin Azamuddin (2018039)	Introduction (Report) Objectives (Report) Map Design (Report & MATLAB) Robot Model (Report & MATLAB) Verifying Full MATLAB Simulation (MATLAB) Script Debugging (MATLAB) Presentation Preparation Integration Testing
Ian Mohd Azrai Bin Raimin (1921057)	MATLAB GUI Development (MATLAB) Performance Evaluation & Validation (Report & MATLAB) Results Comparison & Metrics Analysis (Report & MATLAB) Documentation Formatting (Report) Limitations & Improvements (Report) Conclusion (Report) Final Report Compilation (Report) Presentation Preparation Integration Testing

(A) APPENDIX

Appendix A: AI Usage Log

AI Tool Used: Gemini (Google)

Purpose of Use: To assist in constructing the initial framework for MATLAB simulation, debugging MATLAB script, and proofreading the final report.

Examples of Key Prompts Used:

1. "Help me design a 30×30 map script for simulation in MATLAB by giving me a template to fill in the parameter values"
2. “What is the reason the GUI map is displaying the y-axis in a flipped orientation?"

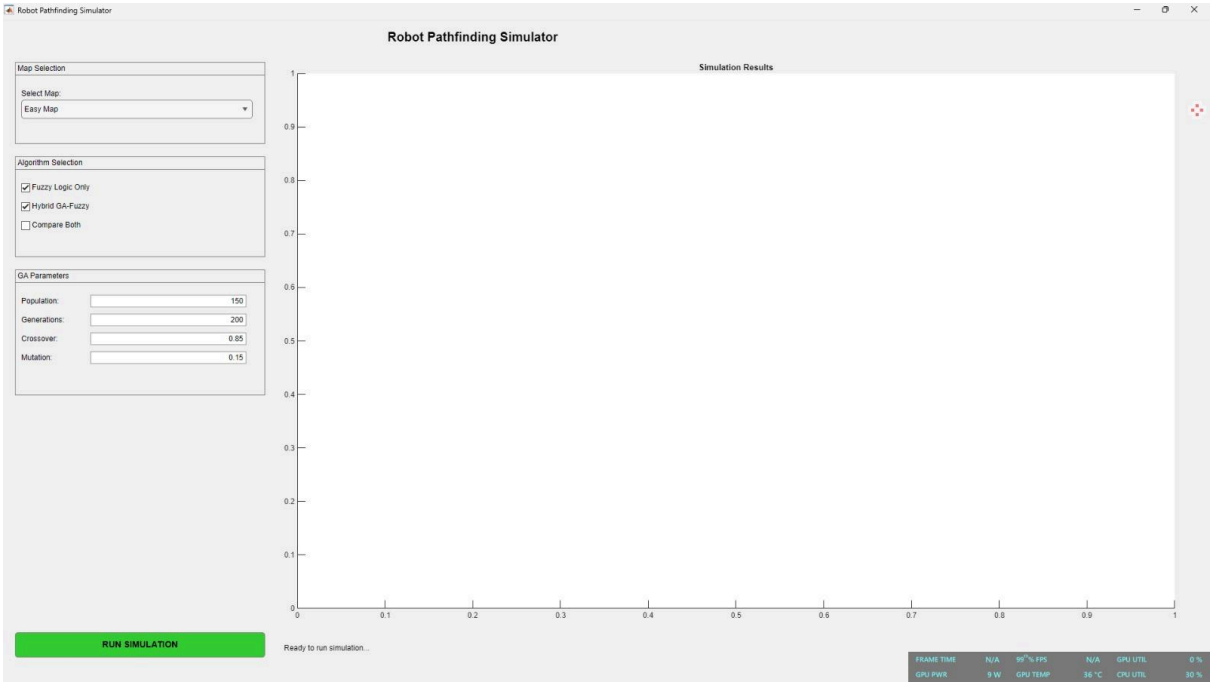
3. “Make me a template/skeleton for a report based on the deliverables asked in the pdf given.”

Verification: All AI-generated code snippets were manually reviewed, tested, and modified by the team. The map coordinates were manually adjusted to create the easy and harp map configurations, and the fitness function weights were tuned to prioritize collision avoidance over speed. We verify that we understand the code implemented and take full responsibility for the final submission.

Appendix B: GitHub Repository Link:

https://github.com/Azzam0A0/MCTA-3371-Computational-Intelligence-Mini-Project

Appendix C: MATLAB GUI Screenshots:



Appendix D: Fuzzy Rule Base:

Goal Distance	Obstacle Distance	Urgency
Near	Close	High
Near	Medium	High
Near	Safe	Moderate
Medium	Close	High

Medium	Medium	Moderate
Medium	Safe	Low
Far	Close	Moderate
Far	Medium	Low
Far	Safe	Low

Appendix E: GA Parameters:

Parameter	Value
Population Size	500
Chromosome Length	300
Generations	100
Elite Count	10
Mutation Rate	0.25
Selection Method	Tournament Selection
Crossover Method	Two-Point Crossover

Appendix F: Sample MATLAB Functions :

createEasyMap()
createHardMap()
runFuzzyNavigation()
runHybridGA()
calculateMetrics()